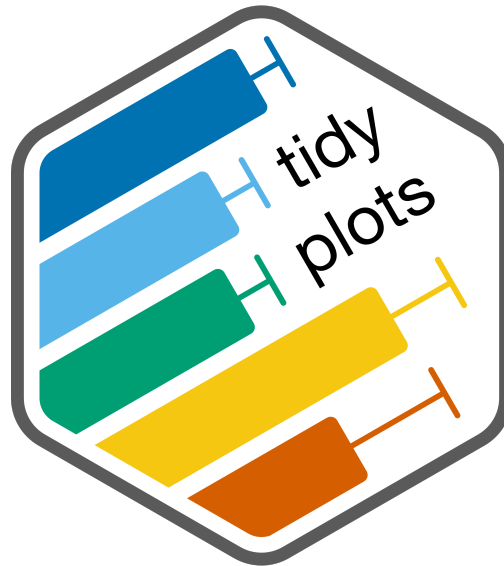


tidyplots: Data Visualisation Made Simple(R)



Associate Professor Dr. Jason Ng

Table of contents

1	What is tidyplots?	3
1.1	Introduction	3
1.2	Some examples	4
2	tidyplots syntax	5
2.1	Workflow Overview	5
2.2	Verbs in tidyplots	5
2.3	An Example	6
3	Case Study (Time Series): COVID-19 in Malaysia	11
3.1	Data Preparation	11

3.2	Line Chart	12
3.3	Stacked Area Chart	13
3.4	Heatmap	15
4	Case Study (Cross-Sectional): GE14	18
4.1	Data Preparation	18
4.2	Histogram	18
4.2.1	Plot histogram	18
4.2.2	Add title and axis labels	19
4.2.3	Change fill colour of bars	20
4.3	Bar Chart	22
4.3.1	Bar chart for counts	22
4.3.2	Bar chart for proportions	23
4.4	Boxplot	27
4.5	Violin plot	30
4.6	Scatter plot	31
4.6.1	Scatter plot	31
4.6.2	Scatter plot with line of best fit	32
4.7	Global vs Local Mapping	33
4.7.1	Global mapping	34
4.7.2	Global mapping: Another example	35
4.8	Facets (Multiple Plots)	36
4.8.1	Split by one variable	36
4.8.2	Split by two variables	38

1 What is tidypLOTS?

1.1 Introduction

- `tidypLOTS` is an R package to generate publication-ready plots for scientific papers.
- Created by Jan Broder Engler. Refer to [Engler \(2025\)](#).
- Goal is to streamline the creation of such publication-ready plots by using a consistent and intuitive syntax that allows you to gradually add, remove and adjust plot components.
- Engler (2025) recognises that although `ggplot2` has its “*utility and power, each of these tools uses a specialized syntax and requires significant coding experience, posing a barrier to adoption by .. scientists*”. He further adds, “*Something as simple as a scatter plot with error bars and a statistical test typically requires a considerable amount of code and intricate knowledge of the plotting tool’s internal workings*”.

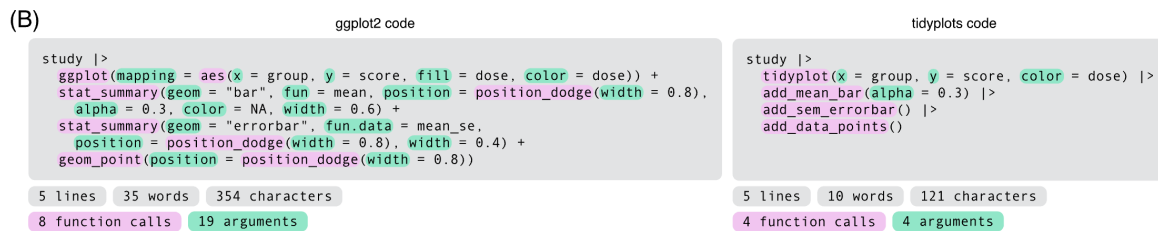


Figure 1: Source: Engler (2025)

- Hence, `tidypLOTS` empowers researchers to leverage automated data visualisation pipelines while minimising required programming skills.

1.2 Some examples

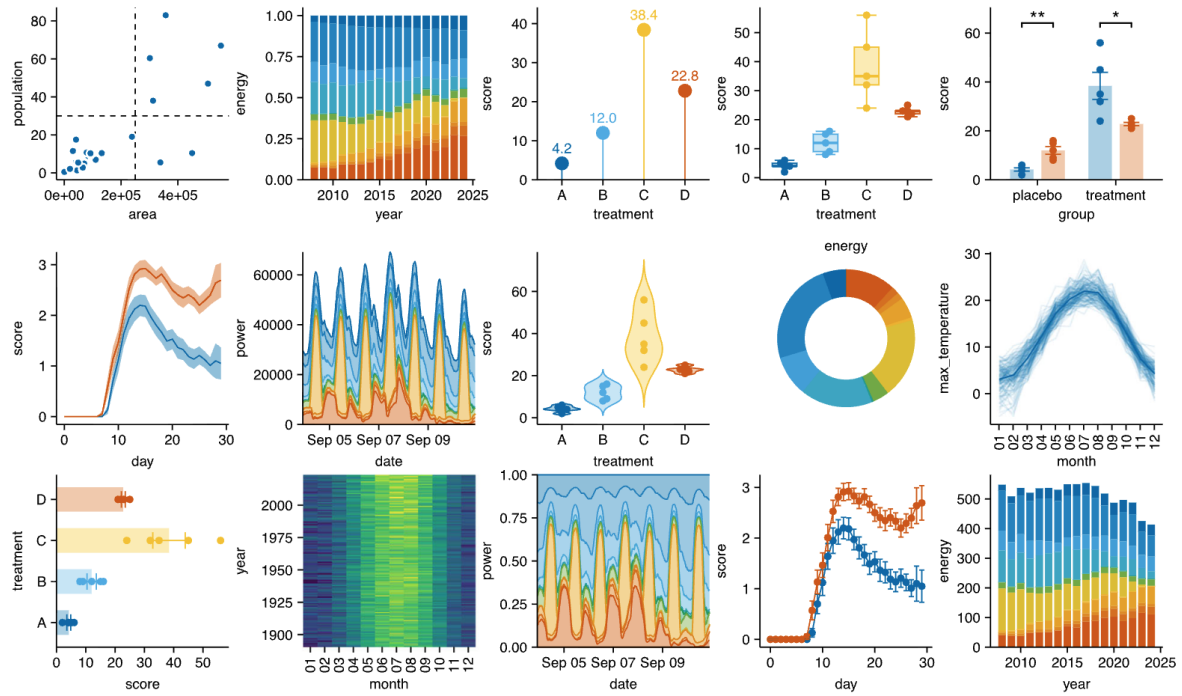


Figure 2: Source: Engler (2025)

2 tidyplots syntax

2.1 Workflow Overview

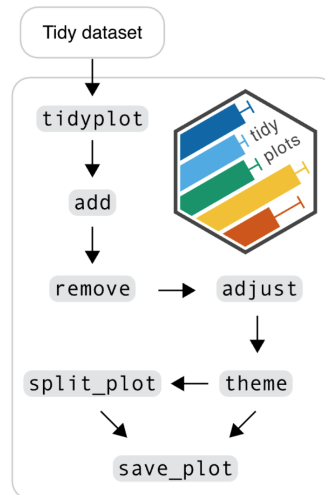


Figure 3: Source: Engler (2025)

2.2 Verbs in tidyplots

- The tidyplots workflow is composed of a series of function calls connected in one pipeline.
- After starting the plot using the `tidyplot()` function, there are three main verbs to construct and modify the plot:
 - (1) `add`
 - (2) `remove`
 - (3) `adjust`
- User can also have the option to choose a different theme (equivalent to `ggplot2`'s `gg_theme`), split the plot into a multiplot layout (equivalent to `ggplot2`'s `facet_grid`) or save it to file.
- The following cheatsheet you a quick overview of the verbs and their associated options. (Click image to enlarge).


tidyplots cheatsheet
 version 0.2.1

Workflow
 dataset → tidyplot → add → remove
 → adjust → theme → save_plot
 → split_plot

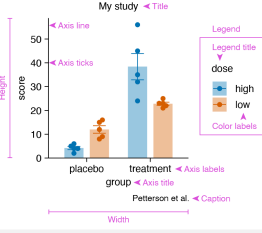
Dataset

	treatment	group	dose	age	sex	score
	<chr>	<chr>	<chr>	<dbl>	<chr>	<dbl>
A	placebo	high	23	female	2	
B	placebo	high	45	male	4	
B	placebo	low	23	female	9	
B	placebo	low	45	male	8	
C	treatment	high	23	female	32	
C	treatment	high	45	male	35	
D	treatment	low	23	female	23	
D	treatment	low	45	male	25	

Code

```

study |>
  tidyplot(x = group, y = score, color = dose) |>
  add_mean_bar(alpha = 0.4) |>
  add_sem_errorbar() |>
  add_data_points(beeswarm()) |>
  add_title("My study") |>
  add_caption("Patterson et al.")
  
```

Plot


Links
[website](#) [docs](#) [help](#) [use-cases](#)
 Copyright © 2025 Jan Brouwer Engler

Add

Raw data
[link](#)
 add_data_ → points() → labels()
 Example: add_data_points()

Summary statistics
[link](#)
 add_count_ → bar() → dash() → dot() → value() → line() → area()
 add_sum_ →
 add_mean_ →
 add_median_ →
 Examples: add_count_bar(), add_sum_line(), add_mean_value()

Dispersion
[link](#)
 add_sem_ → errorbar() → ribbon()
 add_sd_ →
 add_range_ →
 add_ci95_ →
 Example: add_sem_errorbar()

Comparison
[link](#)
 add_test_ → pvalue() → asterisks()
 Example: add_test_pvalue()

Distribution
[link](#)
 add_boxplot() → add_violin()

Proportion
[link](#)
 add_barstack() → add_pie() → add_areastack() → add_donut()

Annotation
[link](#)
 add_title() add_annotation_text()
 add_caption() add_annotation_rectangle()
 add_annotation_line()
 add_reference_lines()

ggplot2 code
[link](#)
 add() → add(ggplot2::geom_point())

More
 add_heatmap() add_curvefit() add_histogram()

Adjust
[link](#)
Plot
 adjust_size() adjust_description()
 adjust_colors() adjust_padding()
 adjust_font() adjust_legend_title()
 adjust_legend_position()

Axes
[link](#)
 adjust_x_axis() adjust_y_axis()
 adjust_x_axis_title() adjust_y_axis_title()

Data labels
[link](#)
 rename_ → color_labels() → x_axis_labels() → y_axis_labels() → reverse()

Examples:
 rename_color_labels()
 sort_x_axis_labels()
 reorder_y_axis_labels()

Remove
[link](#)
Axes
 remove_ → x_axis_ → y_axis_ → line() → ticks() → labels() → title()
 Examples: remove_x_axis(), remove_x_axis_title()

More
 remove_title() remove_legend()
 remove_caption() remove_legend_title()
 remove_padding()

More
[link](#)
Color schemes
 colors_discrete_ → friendly, seaside, apple
 colors_continuous_ → viridis, plasma, mako
 colors_diverging_ → blue2brown, blue2red, spectral

Split plot
[link](#)
 split_plot(by, ncol, nrow, ...)

Output
[link](#)
 save_plot("my_plot.pdf")
 view_plot()

Datasets
[link](#)
 animals, climate, dinosaurs, distributions, energy, energy_week, gene_expression, spendings, study, time_course

2.3 An Example

- We'll use the `study` data set from the `tidyplots` package and follow the example from Engler (2025) to demonstrate the use of `tidyplots`.
- The `study` data set includes a treatment and placebo group at two different treatment doses and a score measuring treatment success.

```
library(tidyplots)
```

```
head(study)
```

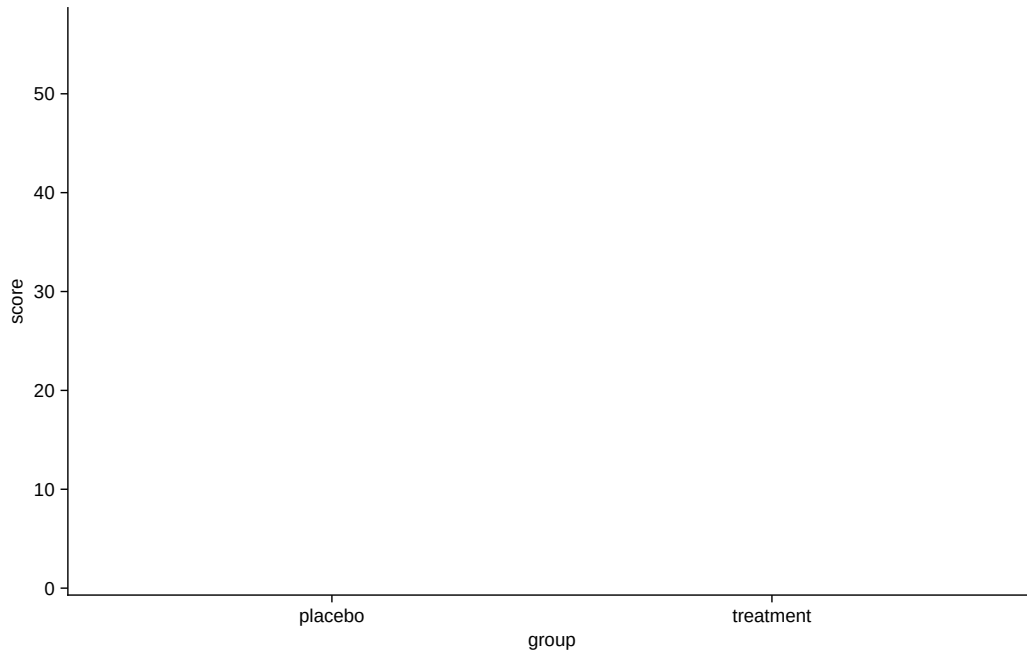
```

# A tibble: 6 x 7
  treatment group dose participant age sex score
  <chr>      <chr> <chr> <chr>      <dbl> <chr> <dbl>
1 A         placebo high  p01         23 female 2
2 A         placebo high  p02         45 male 4
3 A         placebo high  p03         32 female 5
  
```

4	A	placebo	high	p04	37	male	4
5	A	placebo	high	p05	24	female	6
6	B	placebo	low	p06	23	female	9

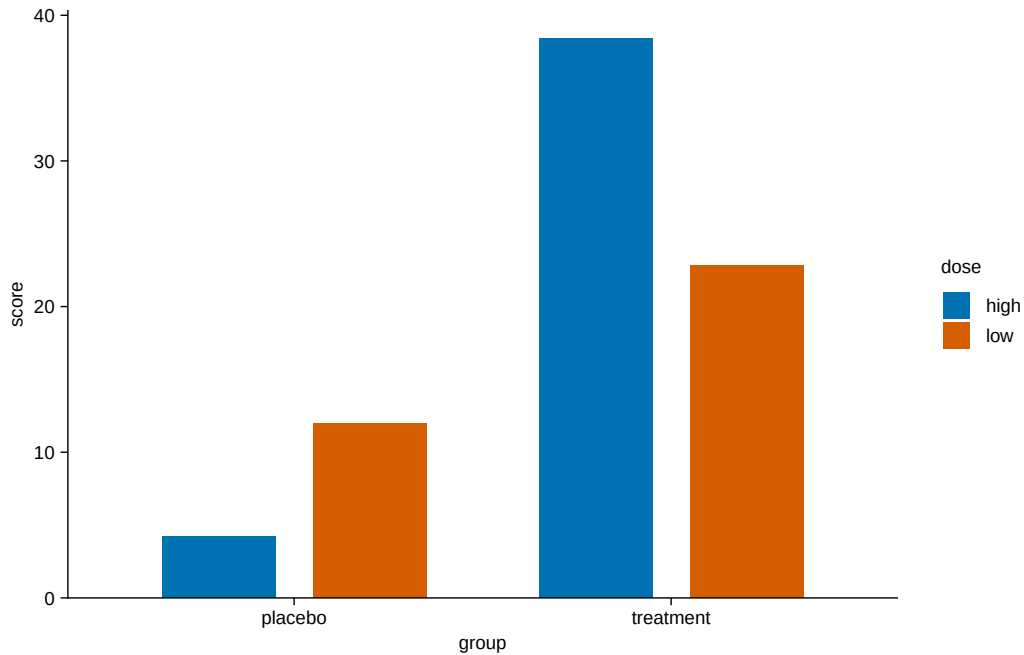
- We'll define x-axis to be **group**, y-axis to be **score** and colour by **dose**.

```
study %>%
  tidyplot(x = group, y = score, colour = dose)
```



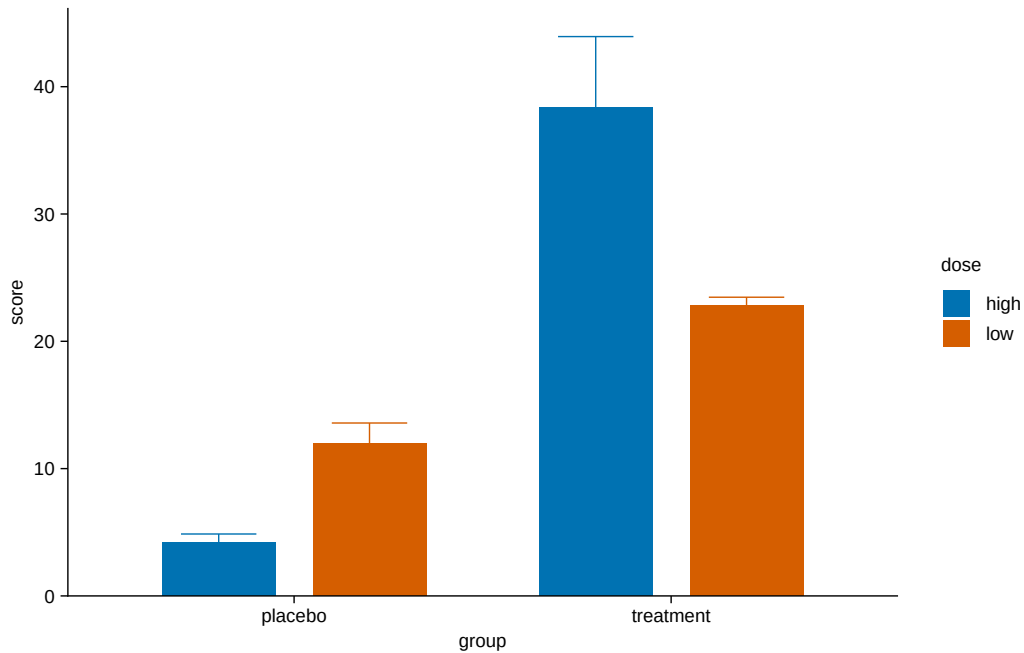
- Next, we *add* the *mean* value of each group represented as a *bar* using the `add_mean_bar()` function. Unlike the `ggplot2` function where we use the `+` operator, we continue to use the `%>%` or `|>` operator.

```
study %>%
  tidyplot(x = group, y = score, colour = dose) %>%
  add_mean_bar()
```



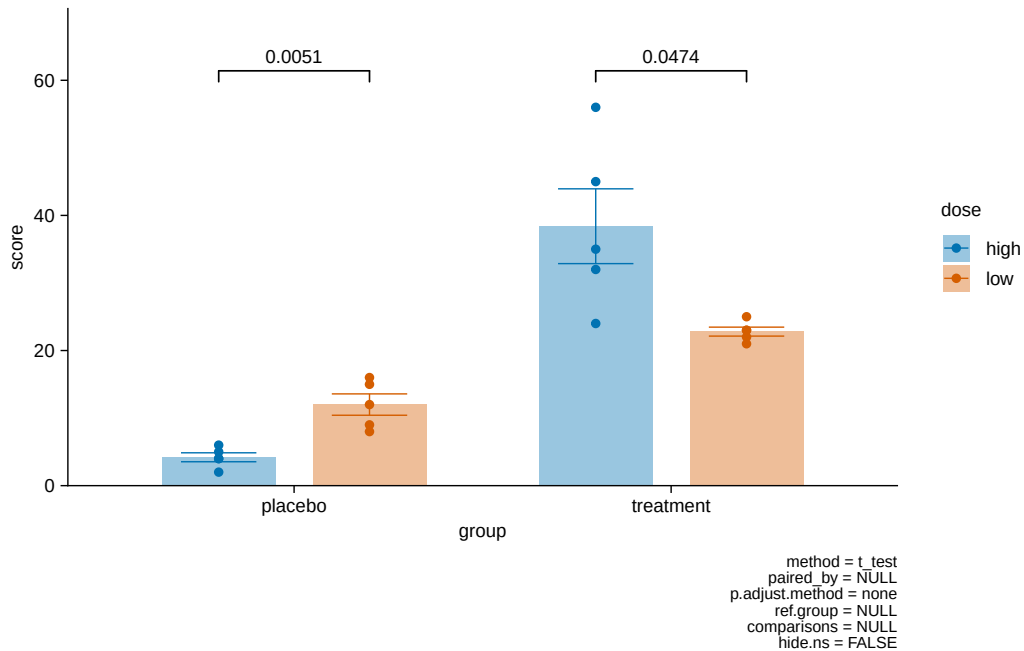
- This exemplifies the general scheme in which the function names
 - (i) start with an action verb (e.g. add),
 - (ii) followed by the statistical entity (e.g. mean),
 - (iii) followed by the graphical representation of the entity (e.g. bar)
- To add the standard error of the mean (sem) as an error bar, we use the function `add_sem_errorbar`:

```
study %>%
  tidyplot(x = group, y = score, colour = dose) %>%
  add_mean_bar() %>%
  add_sem_errorbar()
```



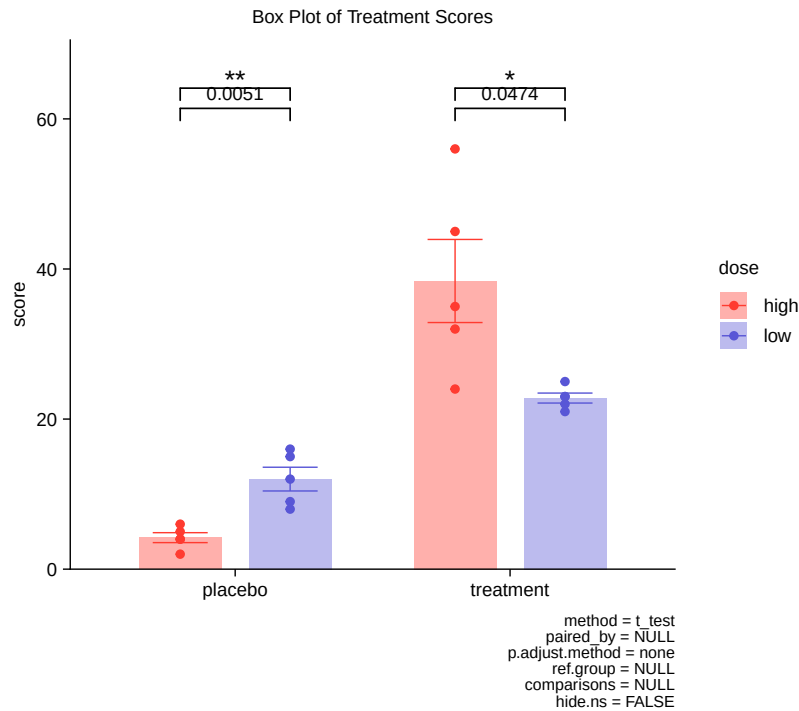
- We'll continue to add the data as data points, and also add a statistical test as p values (we'll also change the opacity of the barchart by using the `alpha` argument:

```
study %>%
  tidyplot(x = group, y = score, colour = dose) %>%
  add_mean_bar(alpha = 0.4) %>%
  add_sem_errorbar() %>%
  add_data_points() %>%
  add_test_pvalue()
```



- In the next step, the user can remove unwanted plot elements and adjust the appearance of the plot, including colours, font, legend, titles, size, padding and axes. For example,

```
study %>%
  tidyplot(x = group, y = score, colour = dose) %>%
  add_mean_bar(alpha = 0.4) %>%
  add_sem_errorbar() %>%
  add_data_points() %>%
  add_test_pvalue() %>%
  add_test_asterisks(hide_info = T) %>%
  add_title(title = "Box Plot of Treatment Scores") %>%
  remove_x_axis_title() %>%
  adjust_size(width = 80, height = 70) %>%
  adjust_colors(colors_discrete_apple)
```



3 Case Study (Time Series): COVID-19 in Malaysia

3.1 Data Preparation

- We'll directly import the daily COVID-19 data for each state and Federal Territory from the [Ministry of Health's Github](#) page.
- Next, we prepare our data by converting the daily cases into monthly cases.

```
covid <- read_csv(
  "https://raw.githubusercontent.com/MoH-Malaysia/covid19-public/refs/heads/main/epidemic/cases.csv"
)

covid <- covid %>%
  mutate(year = year(date),
         month = month(date, label = TRUE))

covid_monthly <- covid %>%
  group_by(state, year, month) %>%
  summarise(cases_new_state = sum(cases_new)) %>%
  mutate(date = make_date(year, month))

view(covid_monthly)
```

3.2 Line Chart

- We compare the `ggplot2` and `tidyplots` codes used to plot (i) all-in-one plot and (ii) multiple plots in one.
- The associated functions to plot a line chart are `geom_line` and `add_line()` in the `ggplot2` and `tidyplots` library packages, respectively.

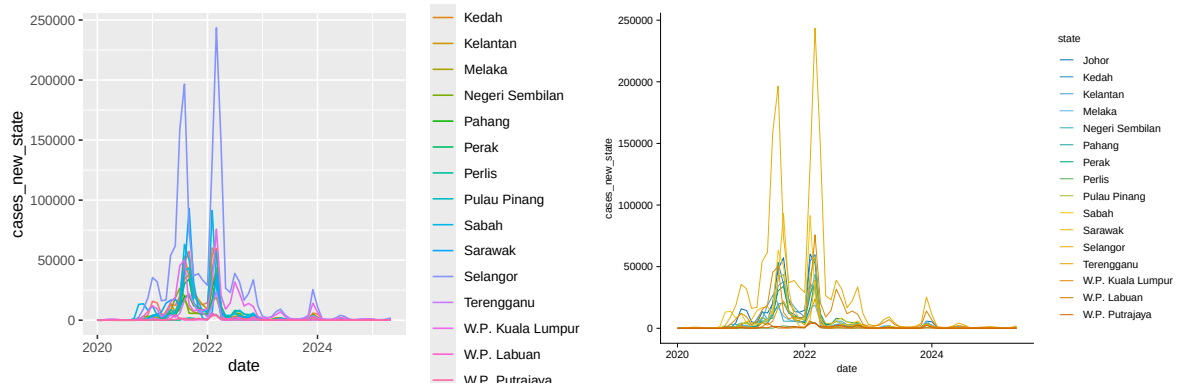
All-in-one plot

```
# ggplot2
covid_monthly %>%
  ggplot(aes(x = date, y = cases_new_state, col = state)) +
  geom_line()

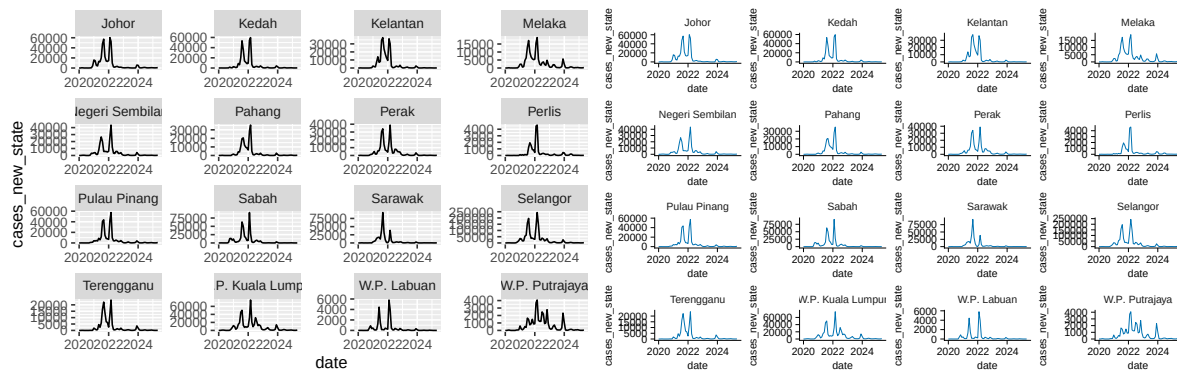
# tidyplots
covid_monthly %>%
  tidyplot(x = date, y = cases_new_state, color = state) %>%
  add_line()
```

Multiple plots in one

```
# ggplot2
covid_monthly %>%
  ggplot(aes(x = date, y = cases_new_state)) +
  geom_line() +
  facet_wrap(~state, nrow = 4, scales = "free")
```



```
# tidyplots
covid_monthly %>%
  tidyplot(x = date, y = cases_new_state) %>%
  add_line() %>%
  split_plot(by = state, nrow = 4)
```



3.3 Stacked Area Chart

- A stacked area chart is essentially an area chart with multiple categories “stacked” on top of each other, representing either absolute counts or relative proportion values.
- Lets compare the proportion of COVID-19 infections for each state across the years. The associated functions are `geom_area` and `add_areastack_relative()` in the `ggplot2` and `tidyplots` library packages, respectively.

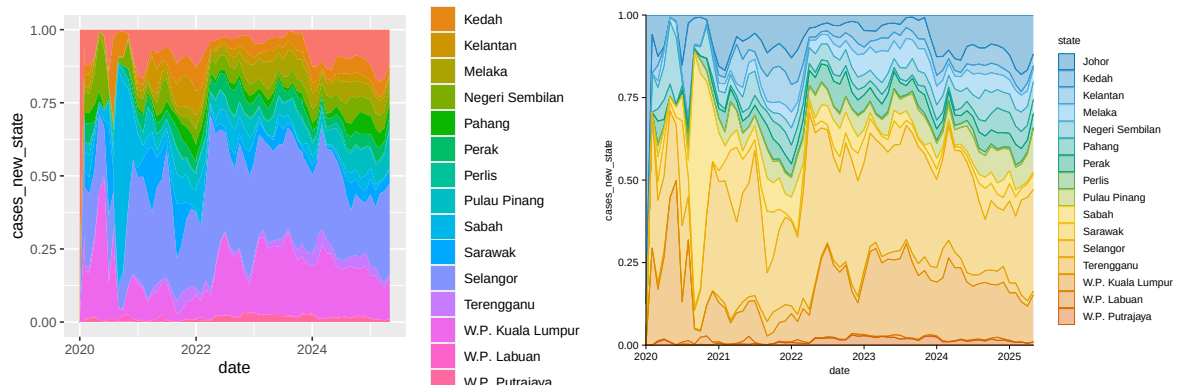
```
#ggplot2
covid_monthly %>%
```

```
ggplot(aes(x = date, y = cases_new_state, fill = state)) +  
  geom_area(position = "fill")
```

```
#tidyplots
```

```
covid_monthly %>%
```

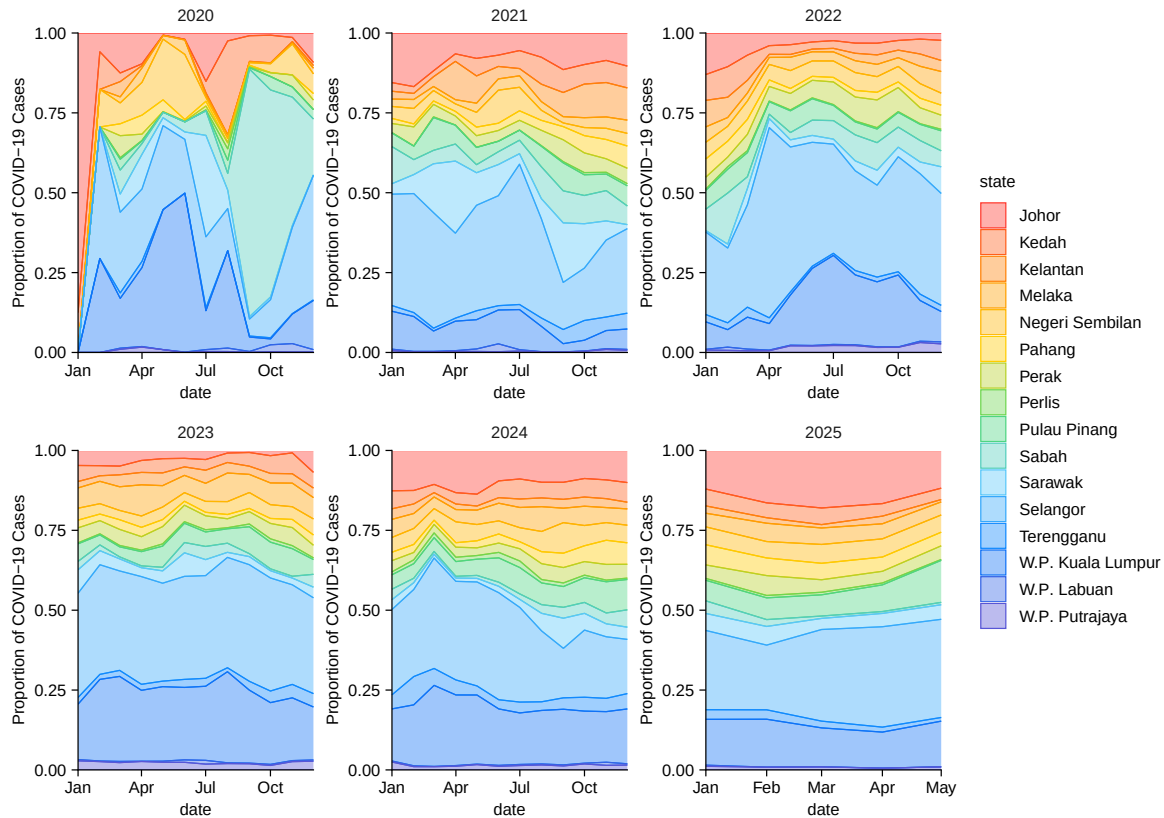
```
  tidyplot(x = date, y = cases_new_state, color = state) %>%  
  add_areastack_relative()
```



- To the tidyplot graph, we'll:
 - change the color of the areas (i.e., states),
 - split them into yearly multiple plots,
 - update the y-axis title

```
covid_monthly %>%
```

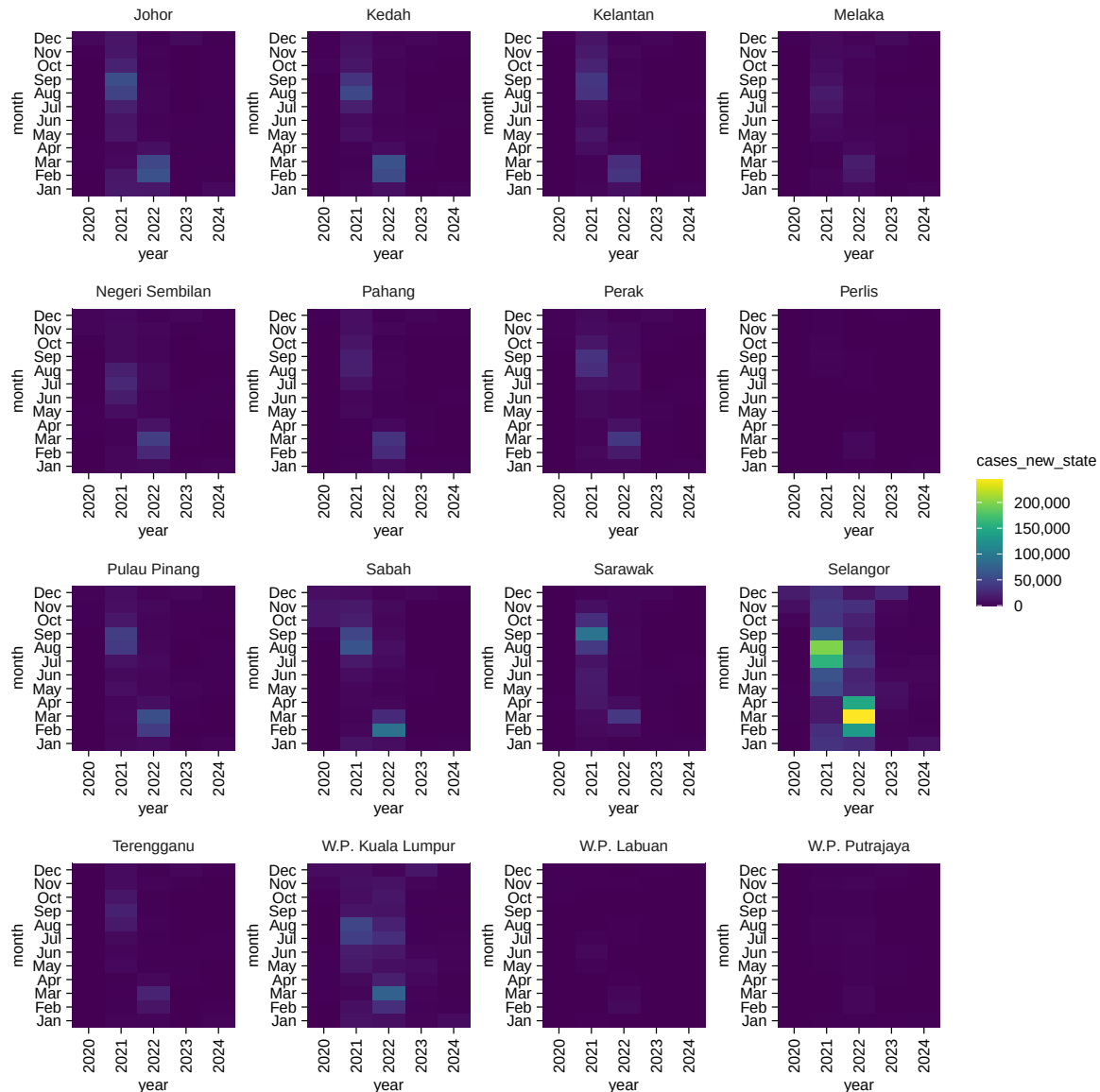
```
  tidyplot(x = date, y = cases_new_state, color = state) %>%  
  add_areastack_relative() %>%  
  adjust_colors(colors_discrete_apple) %>%  
  split_plot(by = year) %>%  
  adjust_y_axis_title(title = "Proportion of COVID-19 Cases")
```



3.4 Heatmap

- A heatmap is a data visualisation tool that uses colours to represent the magnitude or intensity of data points, allowing for quick identification of patterns, trends or user behaviour.
- We'll create a heatmap of COVID-19 infections from 2020 to 2024 for each state using the `add_heatmap()` function. That is, we exclude data from 2025. We'll also adjust the size of the graph, and insert commas into the values in our legend.

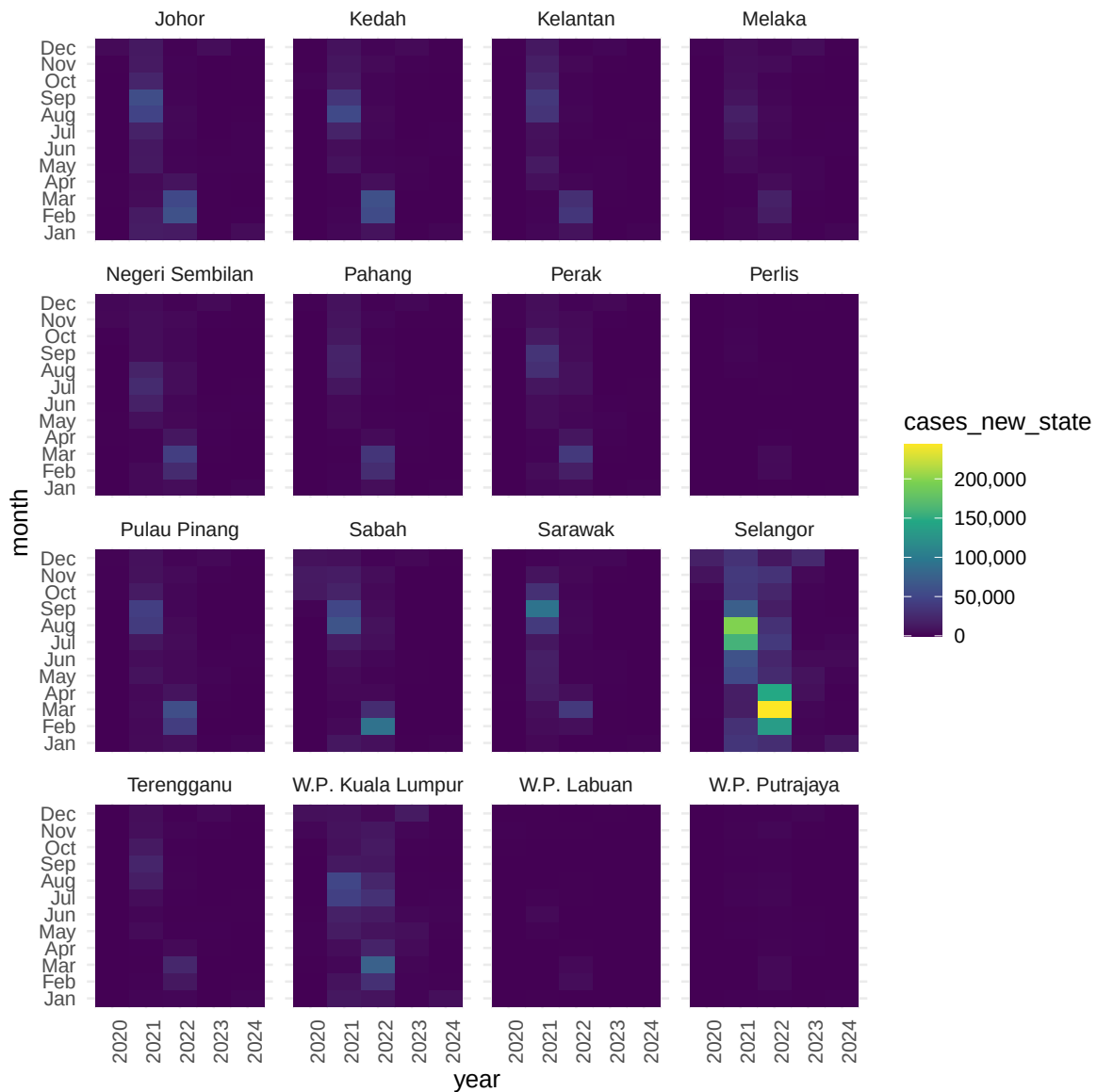
```
covid_monthly %>%
  filter(year < 2025) %>%
  tidypivot(x = year, y = month, fill = cases_new_state) %>%
  add_heatmap() %>%
  split_plot(by = state) %>%
  adjust_colors(labels = scales::label_comma())
```



- The equivalent code of doing this using the `ggplot2` package is:

```
covid_monthly %>%
  filter(year < 2025) %>%
  ggplot(aes(x = year, y = month, fill = cases_new_state)) +
  geom_tile() +
  facet_wrap(~state, nrow = 4) +
  scale_fill_viridis_c(labels = scales::label_comma()) +
```

```
theme_minimal() +  
theme(axis.text.x = element_text(angle = 90))
```



- Compared to the `tidyplots` code chunk, for the `ggplot2` code chunk, we have to manually specify the `gg_theme` and rotate the x-axis labels. Using the `scale_fill_viridis_c` function to insert the commas in the values legend is also not intuitive.

4 Case Study (Cross-Sectional): GE14

4.1 Data Preparation

- Following the data visualisation workshop, we will continue to use the GE14 data to demonstrate the use of `tidyplot`.
- We will focus on data from the Malaysia Peninsular, `df.pen`, and replicate the plots using the `tidyplots` function.

```
library(tidyverse)

df <- read.csv("GE14_cleaned.csv")

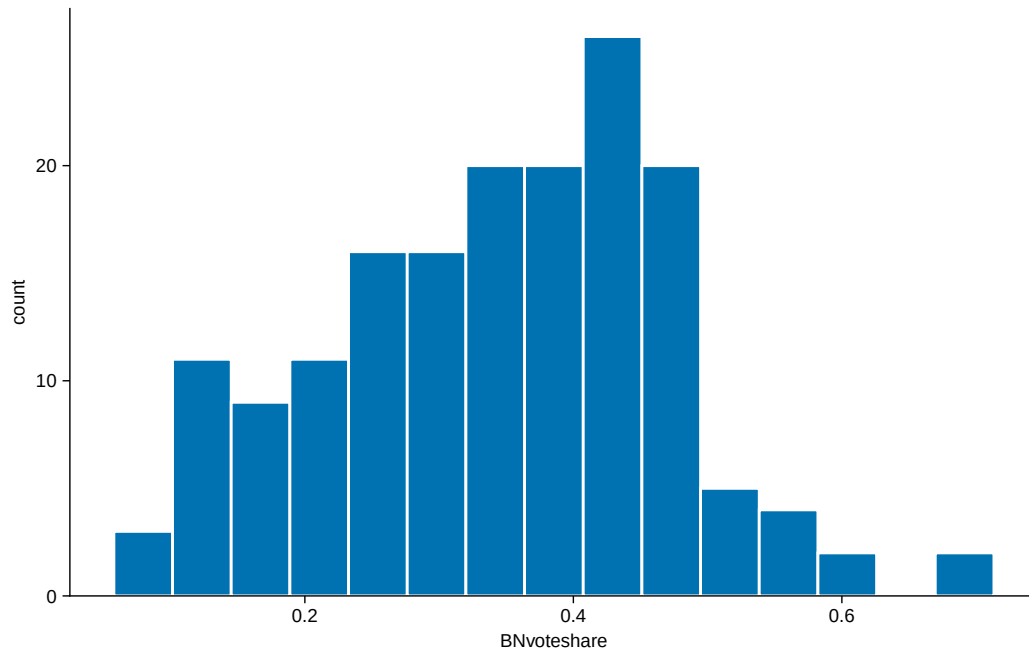
df.pen <- df %>%
  select(Seat.Code, Constituency, State, Malay, BN, PH, Total.Votes, Winner,
         Urban.Semi.U.Rural) %>%
  filter(!State %in% c("Sabah", "Sarawak")) %>%
  rename(Urbanisation = Urban.Semi.U.Rural) %>%
  mutate(Urbanisation = recode_factor(Urbanisation, `1` = "Urban",
                                       `2` = "Semi-urban",
                                       `3` = "Rural"),
         BNvoteshare = BN/Total.Votes,
         State = as.factor(State))
```

4.2 Histogram

4.2.1 Plot histogram

- Plot histogram of the BN vote share. Specify the outline colours of the bar to be `white`.

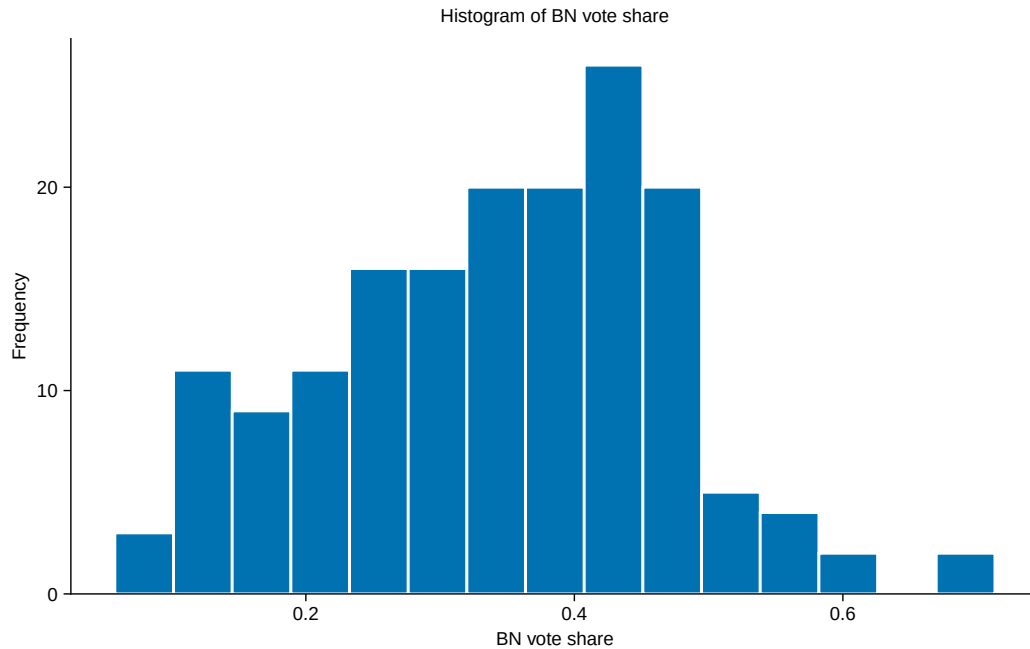
```
df.pen %>%
  tidyplot(x = BNvoteshare) %>%
  add_histogram(bins = 15, colour = "white")
```



4.2.2 Add title and axis labels

- Add title and axis labels using the `add_title()`, `adjust_x_axis_title()` and `adjust_y_axis_title()` functions.

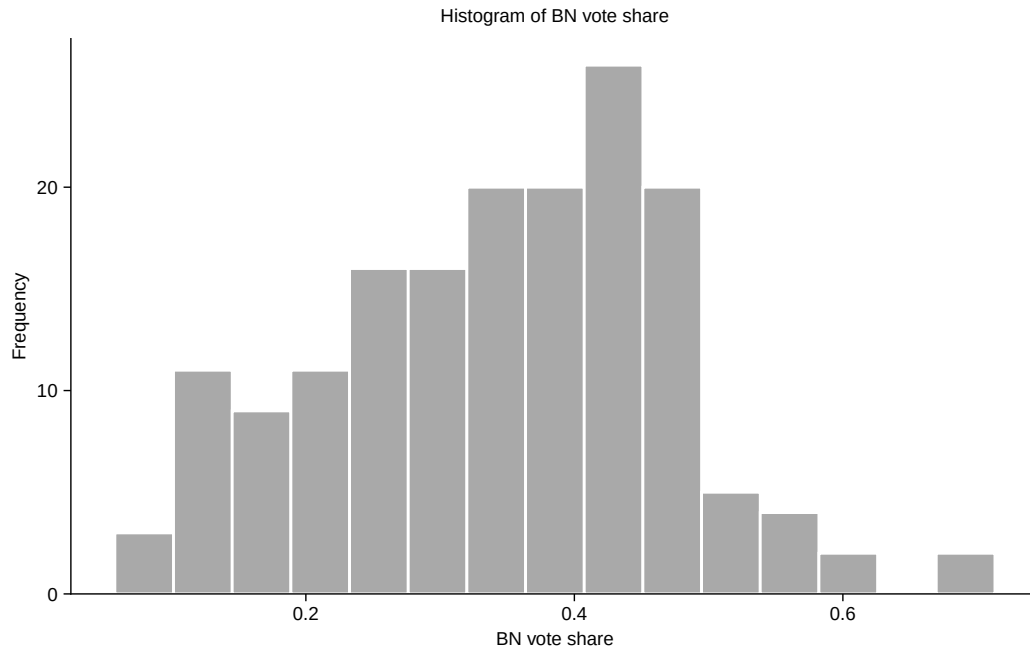
```
df.pen %>%
  tidyplot(x = BNvoteshare) %>%
  add_histogram(bins = 15, colour = "white") %>%
  add_title(title = "Histogram of BN vote share") %>%
  adjust_x_axis_title(title = "BN vote share") %>%
  adjust_y_axis_title(title = "Frequency")
```



4.2.3 Change fill colour of bars

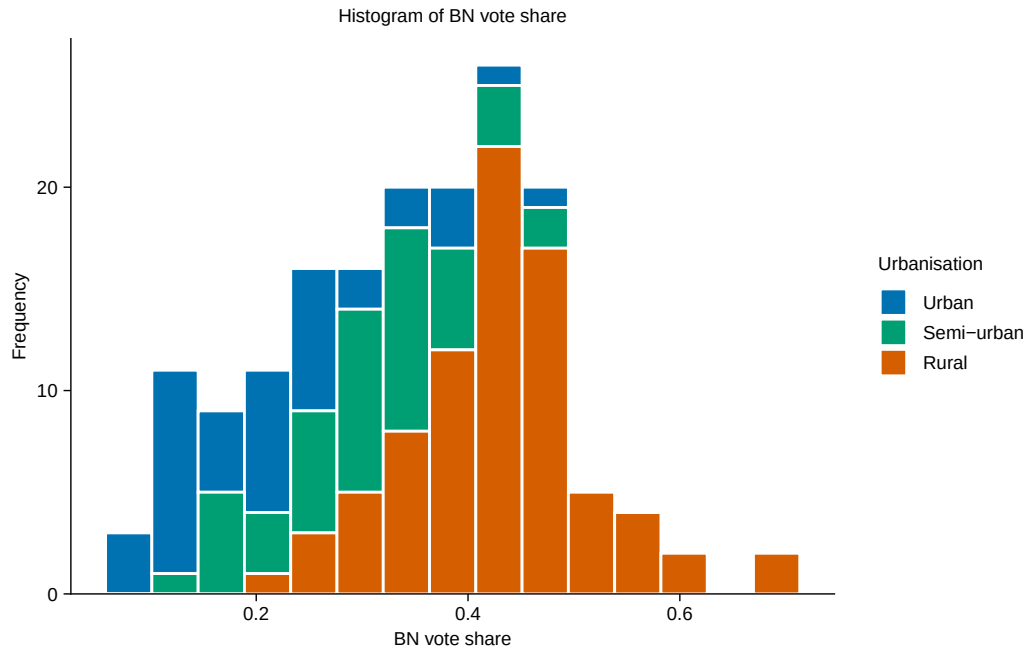
- To change the fill colour of histogram bars, use the `adjust_colors()` function.

```
df.pen %>%
  tidyplot(x = BNvoteshare) %>%
  add_histogram(bins = 15, colour = "white") %>%
  add_title(title = "Histogram of BN vote share") %>%
  adjust_x_axis_title(title = "BN vote share") %>%
  adjust_y_axis_title(title = "Frequency") %>%
  adjust_colors(new_colors = "darkgrey")
```



- To have Urbanisation factor reflected in the histogram.

```
df.pen %>%
  tidyplot(x = BNvoteshare, fill = Urbanisation) %>%
  add_histogram(bins = 15, colour = "white") %>%
  add_title(title = "Histogram of BN vote share") %>%
  adjust_x_axis_title(title = "BN vote share") %>%
  adjust_y_axis_title(title = "Frequency")
```

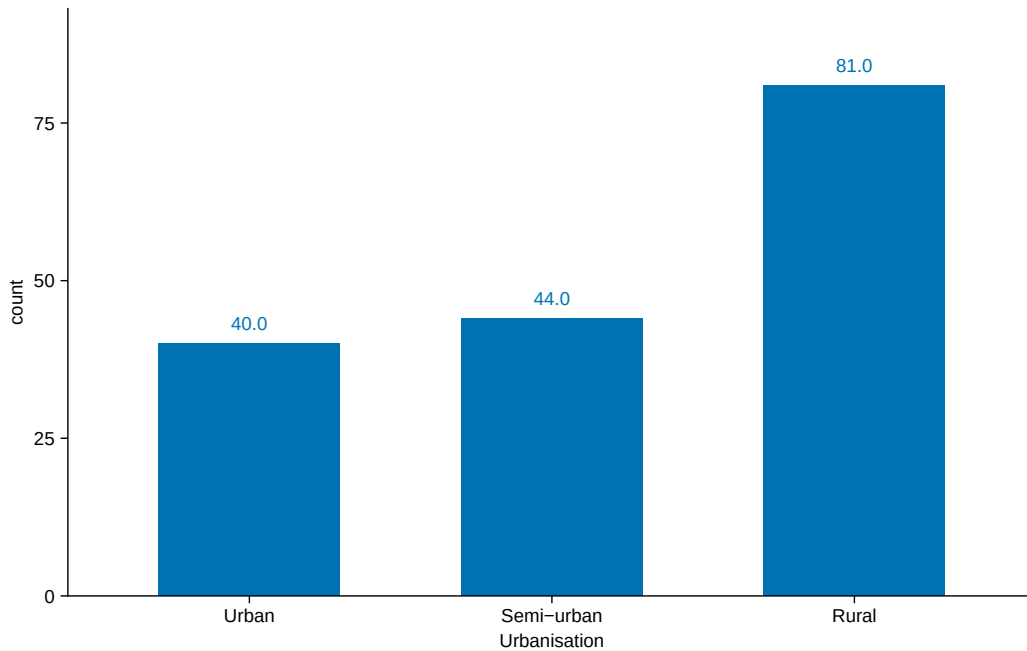


4.3 Bar Chart

4.3.1 Bar chart for counts

- We'll plot bar charts depicting the number of urban, semi-urban and rural seats using the `add_count_bar()` function. We'll also label the number of seats in the chart using the `add_count_value()` function.

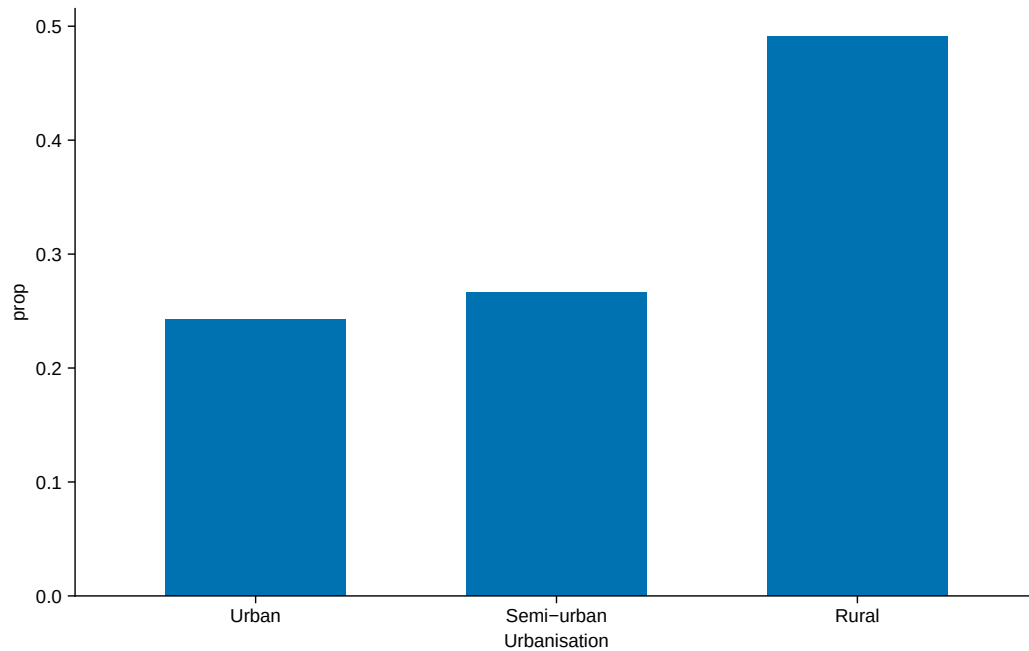
```
df.pen %>%
  tidyplot(x = Urbanisation) %>%
  add_count_bar() %>%
  add_count_value()
```



4.3.2 Bar chart for proportions

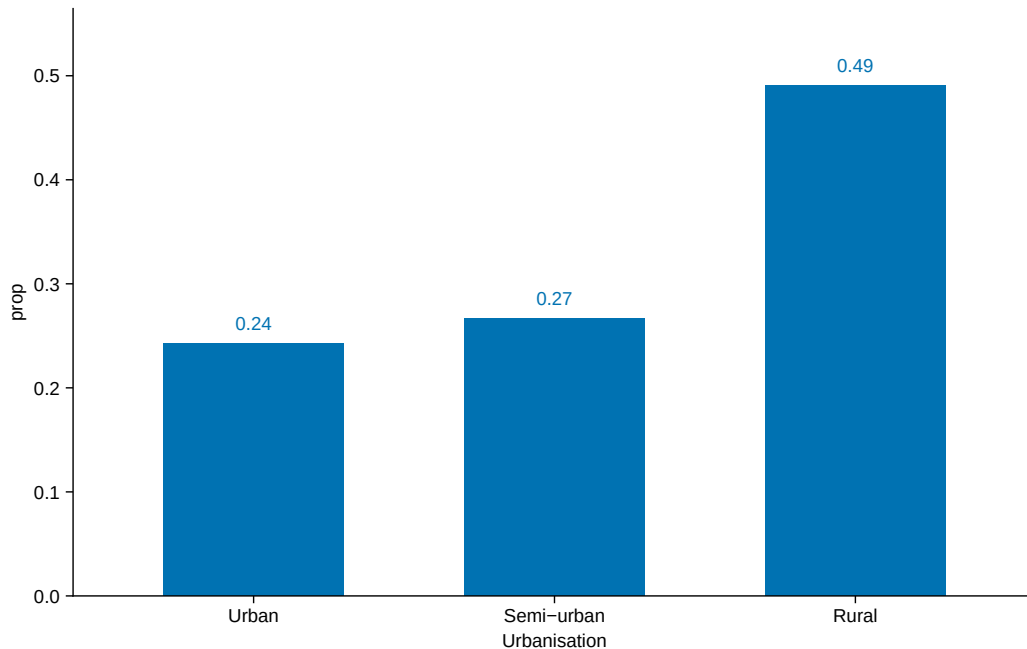
- There is no `tidyplot` function that directly plots the proportions.
- Hence, to plot the proportions, we'll have to first create a proportion variable, `prop`, calculated via the `count` and `mutate` functions before piping it (i.e. `prop`) into the `tidyplot` function with `add_mean_bar()`.

```
df.pen %>%
  count(Urbanisation) %>%
  mutate(prop = n/sum(n)) %>%
  tidyplot(y = prop, x = Urbanisation) %>%
  add_mean_bar()
```



- We can easily add the proportion labels with the `add_mean_value()` function.

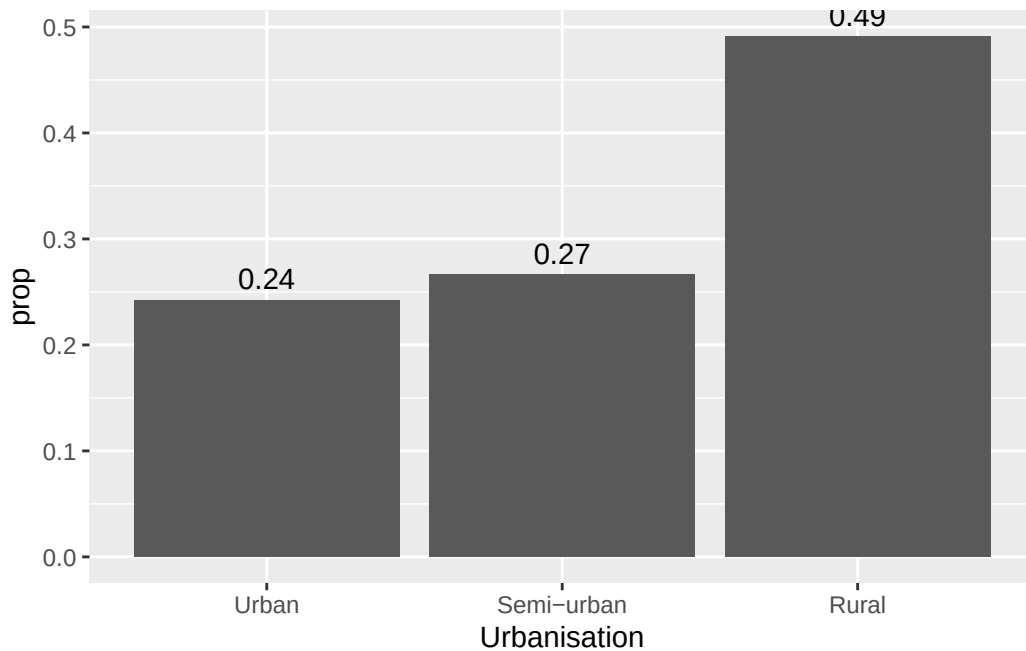
```
df.pen %>%
  count(Urbanisation) %>%
  mutate(prop = n/sum(n)) %>%
  tidypilot(y = prop, x = Urbanisation) %>%
  add_mean_bar() %>%
  add_mean_value(accuracy = 0.01)
```



- In this case, the `ggplot` function would probably be more effective and efficient as it does not require the explicit calculation of the proportions. We would just use `after_stat(prop)` that does the calculation for us under the hood. However, the `tidyplot` function provides an easier option for providing the labels.

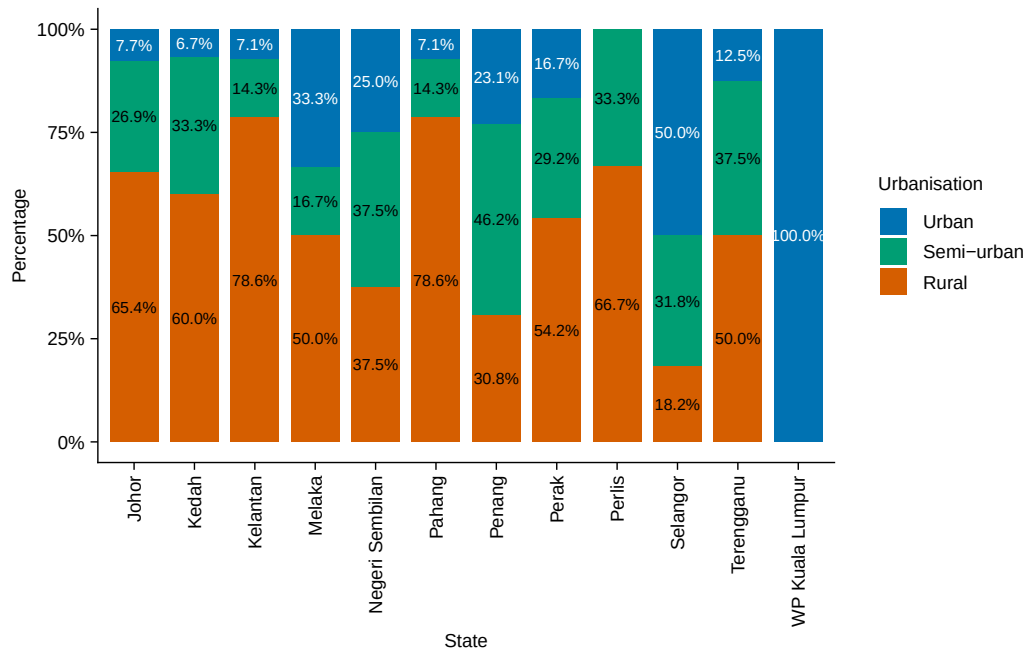
```
library(ggstats) #for the after_stat(prop) function

df.pen %>%
  ggplot(aes(x = Urbanisation,
             y = after_stat(prop),
             group = 1)) +
  geom_bar() +
  geom_text(aes(label = round(after_stat(prop),2)),
            stat = "prop",
            vjust = -.5)
```



- To plot the proportion of Urbanisation seats per state, we use the `add_barstack_relative()` function:

```
df.pen %>%
  tidypilot(x = State, fill = Urbanisation) %>%
  add_barstack_relative() %>%
  adjust_x_axis(rotate_labels = 90) %>%
  adjust_y_axis_title(title = "Percentage") +
  geom_prop_text(size = 2) +
  scale_y_continuous(labels = scales::percent)
```



- Note that we've used the `geom_prop_text()` function from the `ggstats` package for easy labelling. Hence, as per the function's use, we use the `+` operator instead of the `%>%` operator. Same goes with the `scale_y_continuous()` function to have the y-axis labels in percentage.
- The equivalent in `ggplot()` would be:

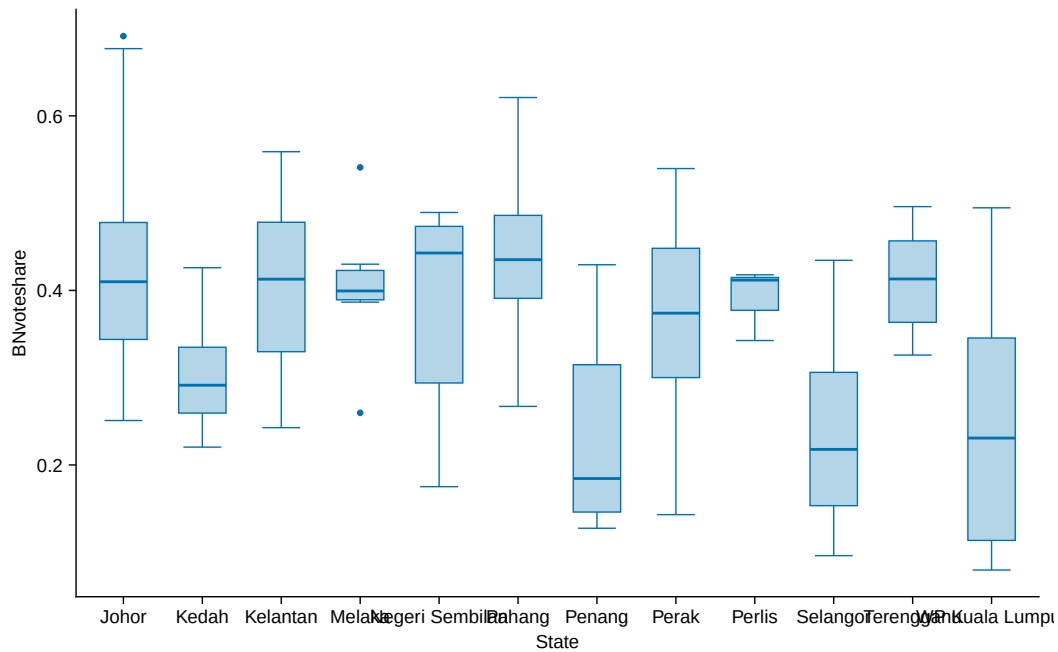
```
ggplot(df.pen, aes(x = State, fill = Urbanisation, by = factor(State))) +
  geom_bar(position = "fill") +
  geom_text(aes(label = scales::percent(after_stat(prop), accuracy = .1)),
    stat = "prop",
    position = position_fill(.5),
    size = 3) +
  scale_fill_brewer(palette = 1) +
  scale_y_continuous(labels = scales::percent) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  labs(y = "Proportion")
```

4.4 Boxplot

- Boxplots are used to display the minimum, maximum, 1st quartile, median and 3rd quartile values of a continuous variable.
- Most useful when comparing these values across categories of a qualitative variable.

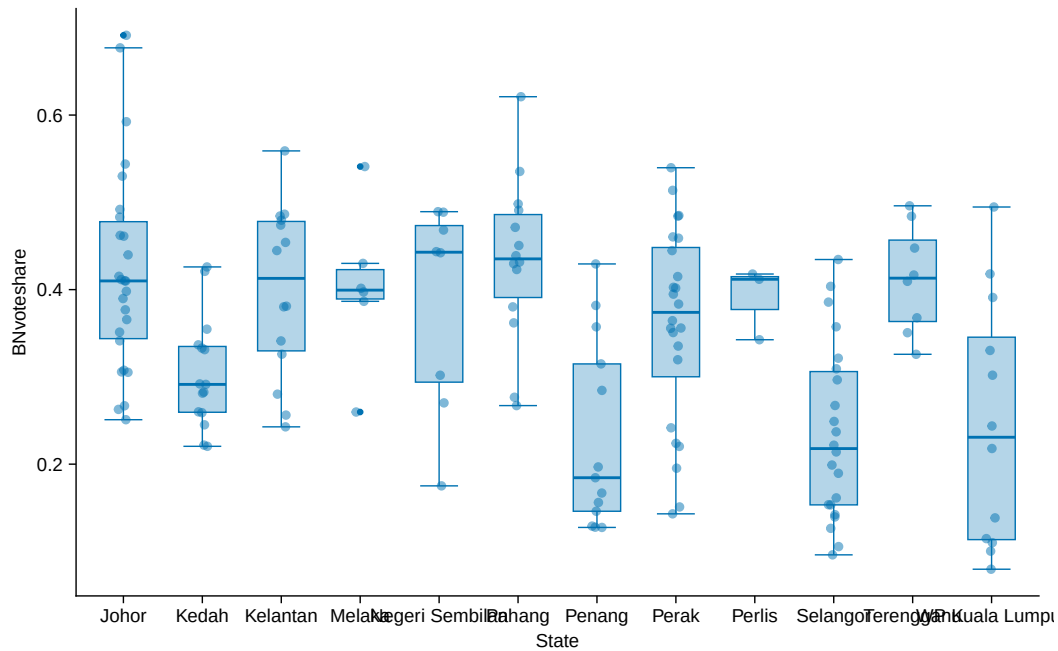
- To view the support for BN across the states,

```
df.pen %>%
  tidyplot(x = State, y = BNvoteshare) %>%
  add_boxplot()
```



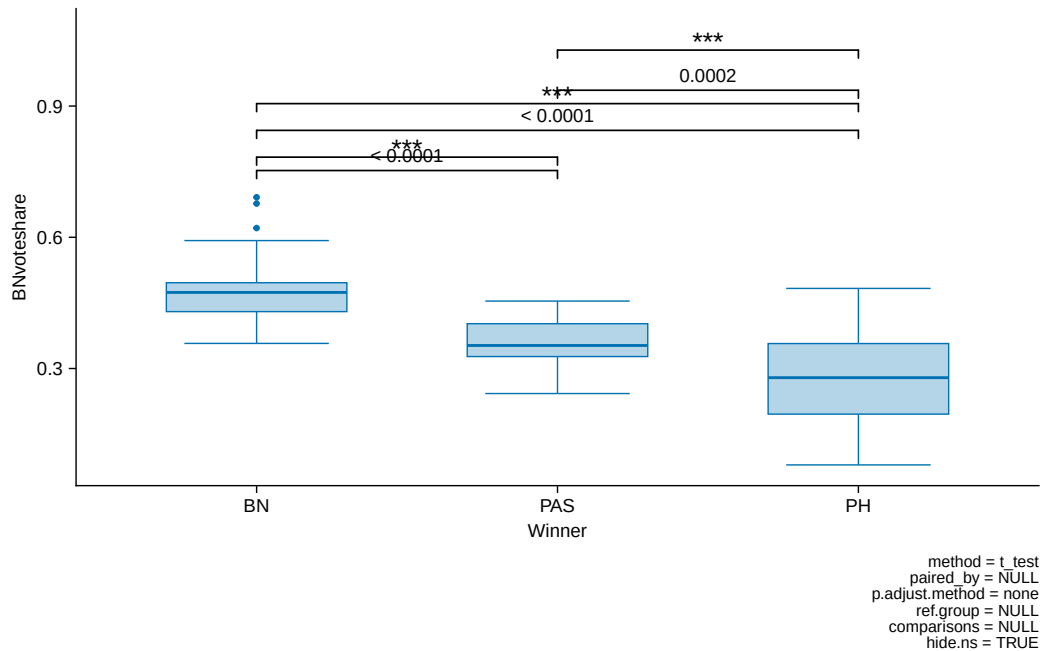
- To add the data points in the boxplot, use the `add_data_points()` or `add_data_points_jitter()` function. Use the `alpha` argument to control the opacity of the data points.

```
df.pen %>%
  tidyplot(x = State, y = BNvoteshare) %>%
  add_boxplot() %>%
  add_data_points_jitter(alpha = 0.5)
```



- Recall the use of the following functions:
 - `add_title(title = ...)`
 - `adjust_x_axis_title(title = ...)`
 - `adjust_y_axis_title(title = ...)`
 - `adjust_x_axis(rotate_labels = 90)` to rotate the x-axis tick labels.
- We can also have statistical test results displayed using `add_test_pvalue`, and `add_test_asterisks` to denote statistical significance.

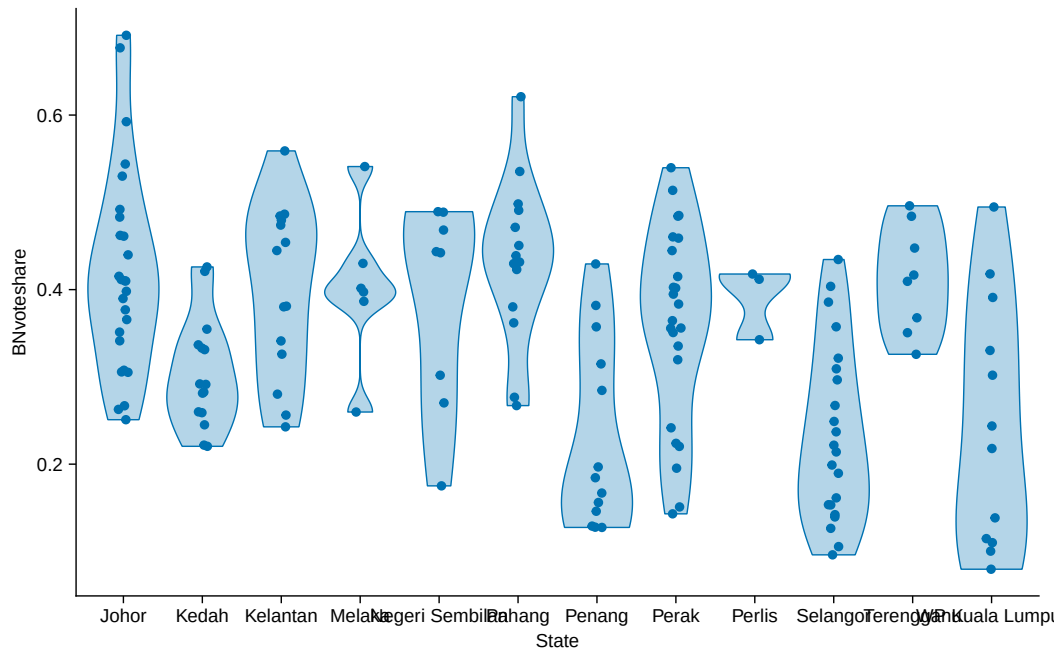
```
df.pen %>%
  tidyplot(x = Winner, y = BNvoteshare) %>%
  add_boxplot() %>%
  add_test_pvalue(p.adjust.method = "bonferroni") %>%
  add_test_asterisks()
```



4.5 Violin plot

- A more 'contemporary' version of the boxplot. Displays continuous distribution of a variable for the different categories of the qualitative variable.

```
df.pen %>%
  tidyplot(x = State, y = BNvoteshare) %>%
  add_violin() %>%
  add_data_points_jitter()
```

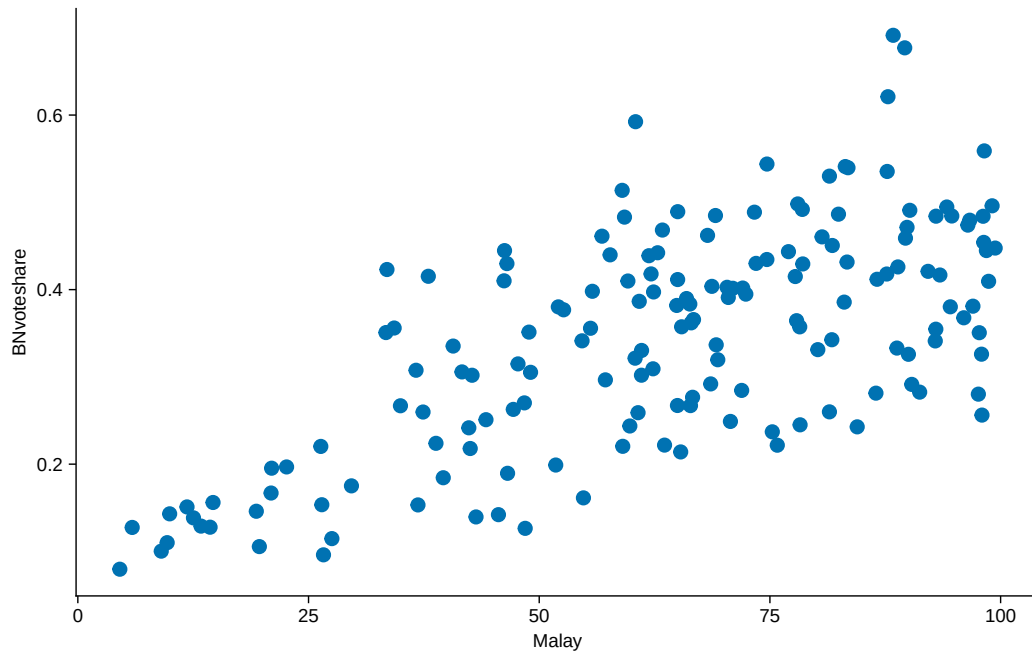


4.6 Scatter plot

4.6.1 Scatter plot

- Scatter plots depict association between 2 continuous variables.
- Lets see the relationship between BN vote share and the constituencies' Malay population share.

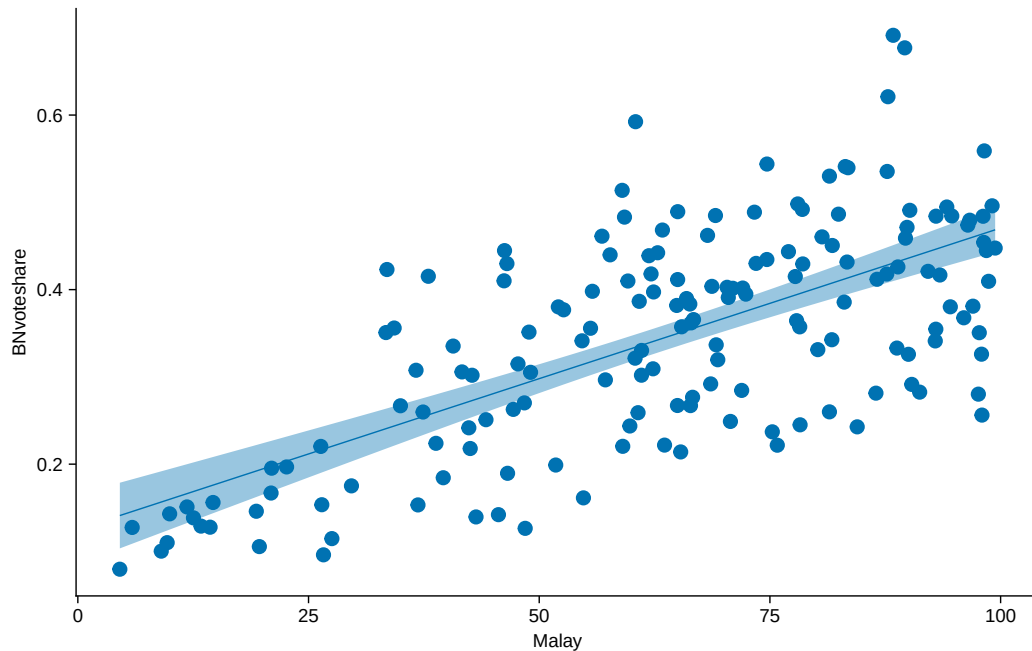
```
df.pen %>%
  tidyplot(x = Malay, y = BNvoteshare) %>%
  add_data_points(size = 2)
```



4.6.2 Scatter plot with line of best fit

- To include the line of best fit, we use the `add_curve_fit()` function and specify the `method` argument as `lm` (linear model). Otherwise, the default `method` is `loess`.

```
df.pen %>%
  tidyplot(x = Malay, y = BNvoteshare) %>%
  add_data_points(size = 2) %>%
  add_curve_fit(method = "lm")
```



4.7 Global vs Local Mapping

- The `ggplot()` function allows flexibility in global and local mapping, allowing for different aesthetics in different layers.
- For example, look at the difference between these 2 code chunks:

```
#Global mapping
ggplot(df.pen, aes(x = Malay,
                   y = BNvoteshare,
                   col = Urbanisation)) +
  geom_point() +
  geom_smooth(method = "lm")

#Local mapping for geom_point
ggplot(df.pen, aes(x = Malay,
                   y = BNvoteshare)) +
  geom_point(aes(col = Urbanisation)) +
  geom_smooth(method = "lm")
```

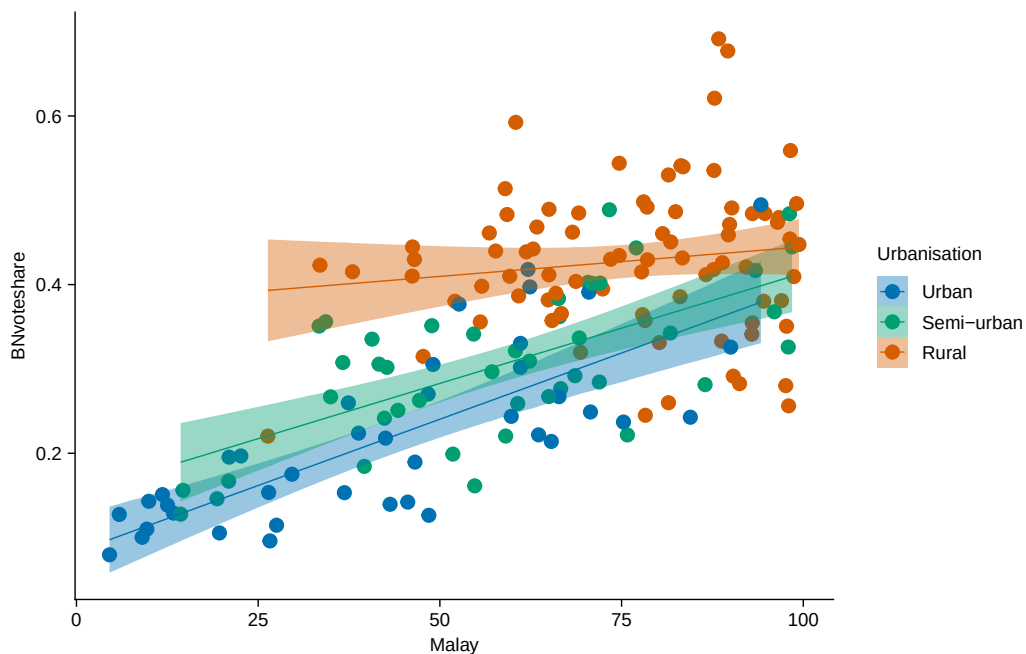
- The `x`, `y` and other visual elements (e.g. `size`, `col`, `fill`) specified in the `tidyplot()` function are regarded as global mapping. However, to have a local mapping for a specific

graphical layer that overwrites the global mapping using the `tidyplots` framework, we will need to `add()` the relevant `geom_*()` function from the `ggplot2` package.

4.7.1 Global mapping

- To have the scatter points and regression lines coloured for each of the urbanisation groups,

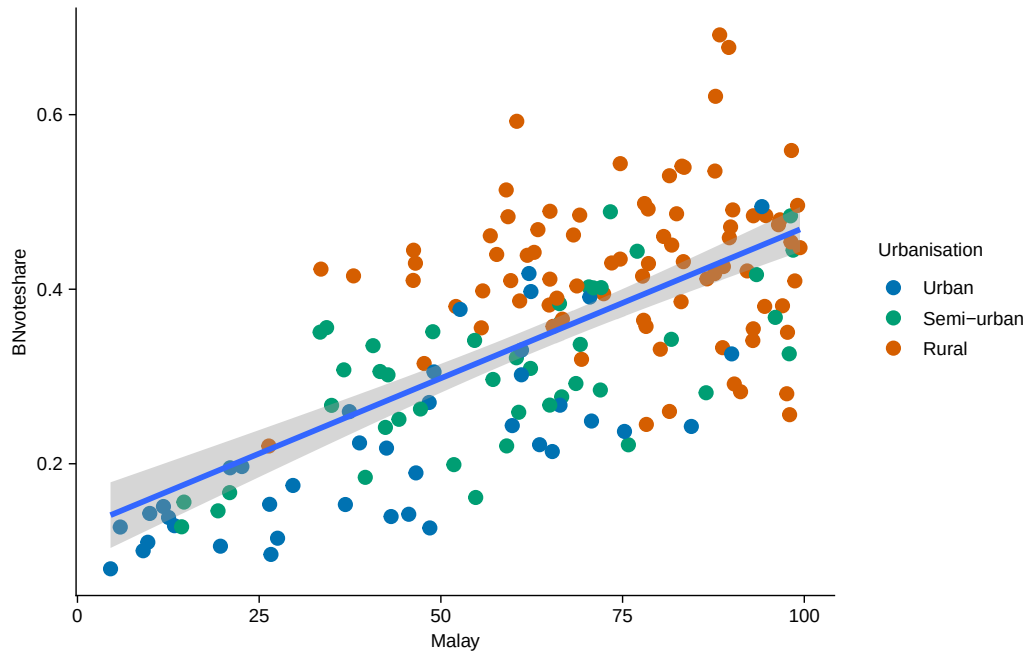
```
df.pen %>%
  tidyplot(x = Malay, y = BNvoteshare, col = Urbanisation) %>%
  add_data_points(size = 2) %>%
  add_curve_fit(method = "lm")
```



- To have the scatter points coloured for each of the urbanisation groups, but an overall regression line for all data points, we add the `geom_smooth()` function from the `ggplot2` package.

```
df.pen %>%
  tidyplot(x = Malay, y = BNvoteshare, color = Urbanisation) %>%
  add_data_points(size = 2) %>%
  add(
    geom_smooth(aes(x = Malay, y = BNvoteshare),
```

```
method = "lm",
inherit.aes = F
))
```



- The parent `tidyplot()` has `color = Urbanisation` — this implicitly groups the data by `Urbanisation`.
- Without `inherit.aes = FALSE`, `geom_smooth()` would inherit that grouping and draw one line per group.
- With `inherit.aes = FALSE`, `geom_smooth()` only sees `aes(x = Malay, y = BNvoteshare)` — no grouping, so it draws one line through all the data.

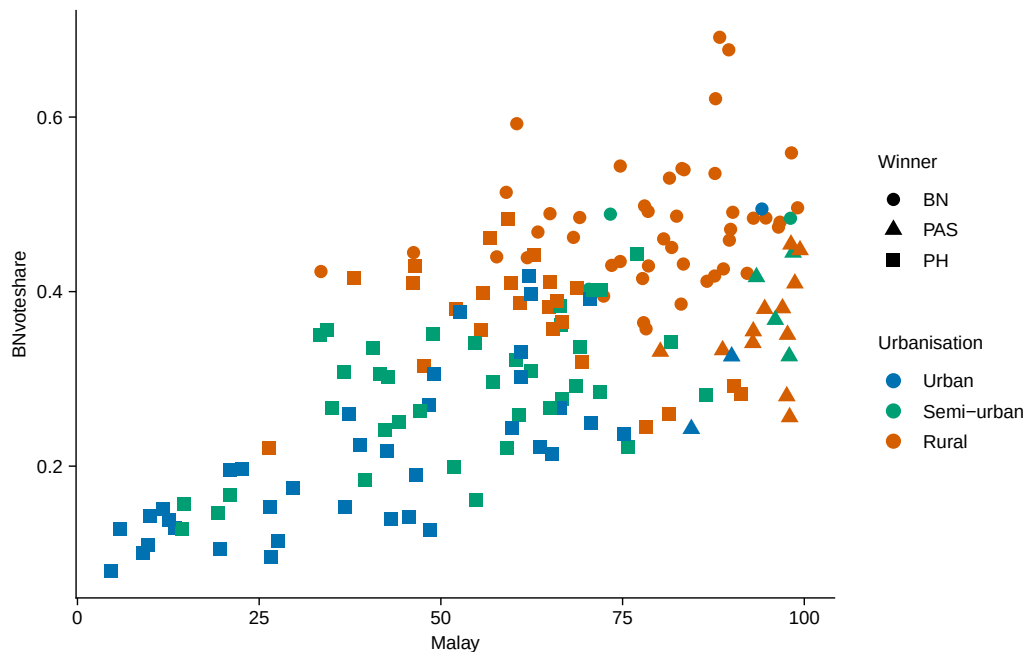
4.7.2 Global mapping: Another example

- Create a scatter plot of `BNvoteshare` against `Malay`, with the points coloured according to the `Urbanisation` groups, and the shape of those points according to the political parties that won those seats.
- Using `ggplot`,

```
ggplot(df.pen, aes(x = Malay, y = BNvoteshare,
                    col = Urbanisation)) +
  geom_point(aes(shape = Winner), size = 2)
```

- Using `tidyplot`,

```
df.pen %>%
  tidyplot(x = Malay, y = BNvoteshare, color = Urbanisation) %>%
  add(
    geom_point(aes(shape = Winner), size = 2)
  )
```



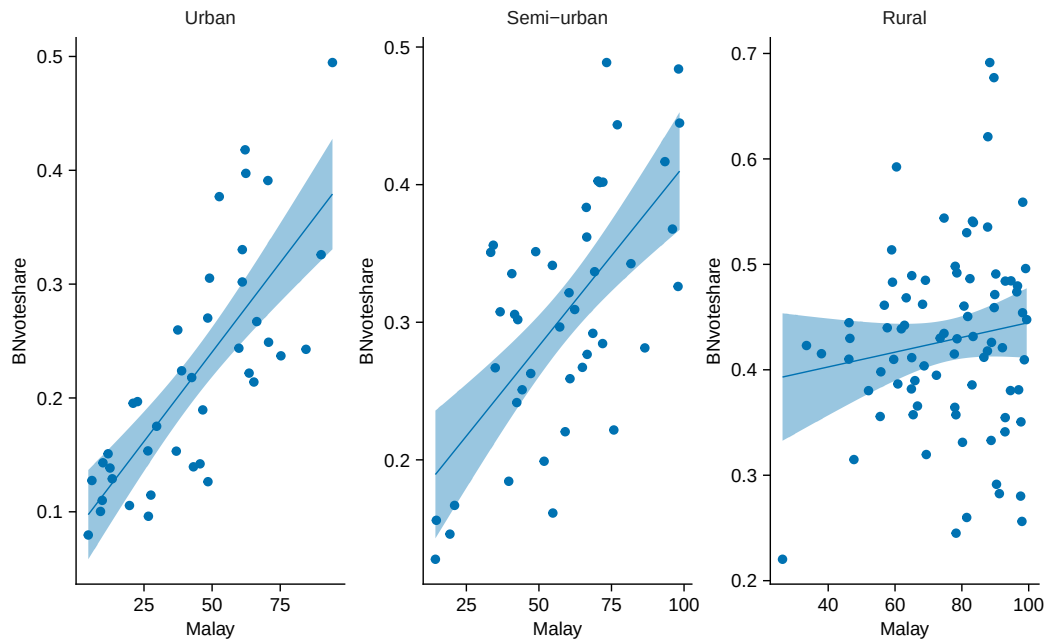
4.8 Facets (Multiple Plots)

- Concept of small multiples.
- The idea of conditioning on the values taken on by one more of the **categorical** variables in a data set.
- Created by adding the layer `facet_grid()` in the `ggplot2` package
- In `tidyplots`, we use `split_plot()`.

4.8.1 Split by one variable

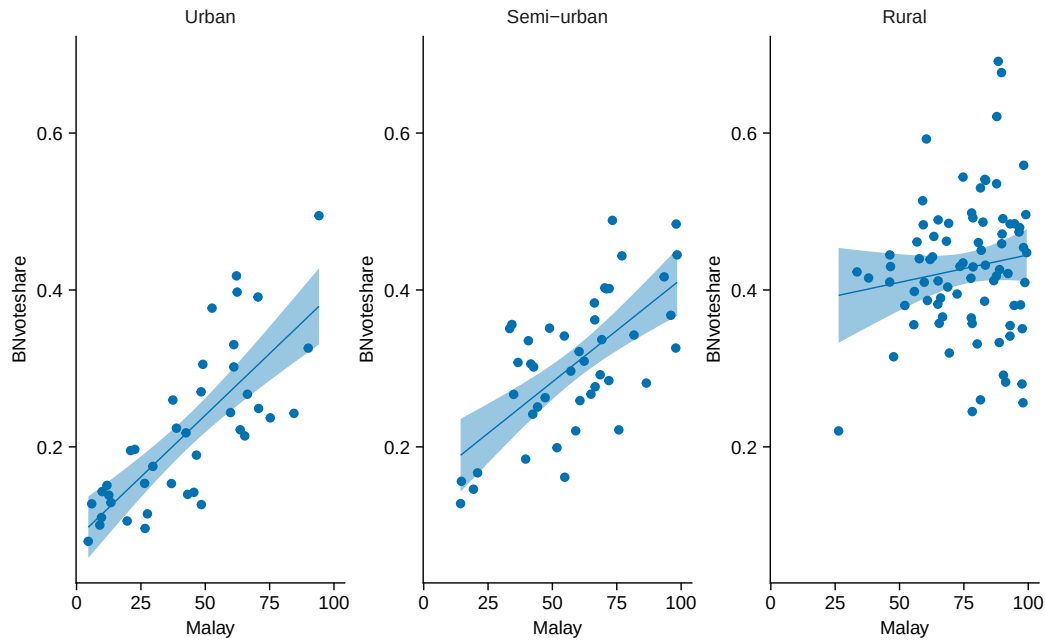
- Display the relationship between Malay population share and BN vote share in each of the urbanisation classification:

```
df.pen %>%
  tidypplot(x = Malay, y = BNvoteshare) %>%
  add_data_points() %>%
  add_curve_fit(method = "lm") %>%
  split_plot(by = Urbanisation)
```



- In `ggplot`, the scales of the y and x axis are fixed by default. The reverse is true for `tidypplot`.
- Similar to the `ggplot`, use the `scales` argument to modify the y and/or x axis. The options are `fixed`, `free_y`, or `free_x`. These options have the same syntax used in `ggplot`.

```
df.pen %>%
  tidypplot(x = Malay, y = BNvoteshare) %>%
  add_data_points() %>%
  add_curve_fit(method = "lm") %>%
  split_plot(by = Urbanisation, scales = "fixed")
```



4.8.2 Split by two variables

- To split the plots by 2 variables, specify the variables to the respective `rows` and `cols` arguments.

```
df.pen %>%
  tidyplot(x = Malay, y = BNvoteshare) %>%
  add_data_points() %>%
  add_curve_fit(method = "lm") %>%
  split_plot(rows = Urbanisation, cols = Winner)
```

